IES FRANCISCO DE GOYA

NOMBRE: Martin Taboada
CURSO: DAM 2

Entornos de Desarrollo: Calidad y Diseño del Software
Ejercicios de Documentacion Javadoc

En el presente documento se presentarán los fragmentos de código con el añadido de Tags y comentarios apropiados para una próxima documentacion para Javadoc pertenecientes al ejercicio planteado en el aula virtual, no se ha optado por subir la documentación directa de javadoc por eficiencia a la hora de mandar un solo documento pdf.

Para este deber, el nombre de autor cambia conforme cada ejercicio y es aleatorio, además, se han utilizado las siguientes etiquetas:

- `@author`
- `@version`
- `@param`
- `@return`
- `@throws`
- `@see`
- `@since`
- `@deprecated`

Ejercicio 1: Interfaz de Servicio

```
/**
 * Interfaz que define las operaciones básicas para la gestión de
productos.
 * Permite consultar, registrar, eliminar y contar los productos
 * disponibles en el sistema.
 *
 * @author Juanito Alimaña
 * @version 1.0
 */
public interface GestorProductos {

    /**
     * Obtiene la lista completa de productos registrados.
     *
     * @return una lista con todos los productos disponibles.
     */
    List<Producto> obtenerTodos();

    /**
     * Busca un producto mediante su identificador único.
     *
```

```
     * @param id identificador del producto que se desea consultar.
     * @return el producto correspondiente al identificador indicado,
     *         o null si no existe.
     */
    Producto obtenerPorId(int id);

    /**
     * Guarda un producto en el sistema.
     * Si el producto ya existe, su información puede actualizarse,
     * dependiendo de la implementación.
     *
     * @param producto objeto Producto que se desea guardar.
     */
    void guardar(Producto producto);

    /**
     * Elimina un producto utilizando su identificador único.
     *
     * @param id identificador del producto que se desea eliminar.
     * @return true si el producto fue eliminado correctamente;
     *         false si no se encontró.
     */
    boolean eliminar(int id);

    /**
     * Obtiene la cantidad total de productos registrados.
     *
     * @return número total de productos almacenados.
     */
    int contarProductos();
}
```

Ejercicio 2: Clase de Excepciones Personalizada

```
/**
 * Excepción personalizada utilizada para representar errores
 * relacionados con las operaciones de gestión de usuarios.
 *
 * Esta clase permite identificar diferentes situaciones de error,
 * como usuarios inexistentes, datos inválidos o usuarios duplicados.
 *
 * @author PerooFlauta
 * @version 1.0
 */
public class ExcepcionUsuario extends RuntimeException {

    /**
     * Construye una nueva excepción relacionada con usuarios.
     *
     * @param mensaje descripción del error producido.
     */
```

```
    public ExcepcionUsuario(String mensaje) {
        super(mensaje);
    }

    /**
     * Excepción que indica que un usuario no ha sido encontrado
     * en el sistema.
     *
     * @author Nombre del estudiante
     * @version 1.0
     */
    public static class UsuarioNoEncontrado extends ExcepcionUsuario {

        /**
         * Construye una excepción para indicar que el usuario
         * solicitado no existe.
         *
         * @param mensaje descripción del error producido.
         */
        public UsuarioNoEncontrado(String mensaje) {
            super(mensaje);
        }
    }

    /**
     * Excepción que indica que los datos proporcionados
     * para un usuario no cumplen con los requisitos establecidos.
     *
     * @author Nombre del estudiante
     * @version 1.0
     */
    public static class DatosInvalidos extends ExcepcionUsuario {

        /**
         * Construye una excepción relacionada con datos de usuario
         * incorrectos o no válidos.
         *
         * @param mensaje descripción del error producido.
         */
        public DatosInvalidos(String mensaje) {
            super(mensaje);
        }
    }

    /**
     * Excepción que indica que se intenta registrar un usuario
     * que ya existe en el sistema.
     *
     * @author Nombre del estudiante
     * @version 1.0
     */
    public static class UsuarioDuplicado extends ExcepcionUsuario {

        /**
```

```
         * Construye una excepción para indicar que el usuario
         * que se intenta registrar ya está registrado.
         *
         * @param mensaje descripción del error producido.
         */
        public UsuarioDuplicado(String mensaje) {
            super(mensaje);
        }
    }
}
```

Ejercicio 3: Método con Parámetros y Excepciones

```
/**
 * Calcula el descuento correspondiente a una compra teniendo
 * en cuenta el precio base, el tipo de cliente y la cantidad
 * de productos adquiridos.
 *
 * El precio base debe ser mayor que cero para realizar
 * correctamente el cálculo.
 *
 * @param precioBase precio original del producto antes de aplicar
 *                   cualquier descuento.
 * @param tipoCliente categoría o clasificación del cliente,
 *                    utilizada para determinar el descuento aplicable.
 * @param cantidad número de unidades adquiridas por el cliente.
 *
 * @return importe final de la compra después de aplicar
 *         el descuento correspondiente.
 *
 * @throws IllegalArgumentException si el precio base es menor
 *                                  o igual a cero.
 */
public BigDecimal calcularDescuento(
        BigDecimal precioBase,
        String tipoCliente,
        int cantidad) {

    if (precioBase.compareTo(BigDecimal.ZERO) <= 0) {
        throw new IllegalArgumentException(
            "El precio debe ser mayor que cero");
    }

    // Cálculo del descuento...
    return precioFinal;
}
```